

Smart Site Alert

Monitoramento de site com alerta por ligação telefônica

1. O que é este serviço

O **Smart Site Alert** é um laboratório self-hosted que **monitora um site e, quando ele cai, liga automaticamente para um telefone (softphone) tocando um aviso de voz**. Tudo roda localmente, em containers Docker, sem depender de serviços externos.

A ideia central:

```
O site parou de responder? -> o telefone toca e uma voz avisa.
```

É útil para qualquer situação em que um e-mail ou notificação no celular passa despercebido, mas uma **ligação** chama a atenção na hora: queda de um servidor crítico, de uma API de produção, de um site institucional, etc.

Por que ligação e não e-mail?

E-mail e push se perdem no meio de centenas de notificações. Uma chamada telefônica força a atenção imediata — é difícil ignorar o telefone tocando. Por isso esse padrão é usado em monitoramento de infraestrutura crítica (NOC, plantão de TI, on-call).

2. Como funciona (arquitetura)

O serviço encadeia quatro peças, cada uma com uma única responsabilidade:

```
[ nginx ]      [ Zabbix ]      [ Asterisk ]      [ Linphone ]
site de      --->  monitora o site --->  central      --->  softphone
exemplo      a cada 30s      telefônica      (toca no fone)
              |
              | (caiu?)      ^
              v              | origina a chamada
              |              | via API (ARI)
dispara a Action -----+
```

Fluxo completo, passo a passo:

1. O **Zabbix** consulta o site (via *Web scenario*) a cada 30 segundos, esperando um HTTP 200.
2. Se o site não responde, o item `web.test.fail` vira `1` e a **trigger "Site DOWN"** entra em estado de problema.
3. A **Action** do Zabbix reage ao problema e chama um **Media type Webhook**.

4. O webhook faz uma requisição HTTP para a **API do Asterisk (ARI)**, pedindo para originar uma chamada.
5. O **Asterisk** liga para o ramal **1000** e, quando atendido, toca o áudio de alerta gravado.
6. O **Linphone** (softphone registrado no ramal 1000) toca no fone de ouvido. Você atende e ouve a voz.

As peças (containers Docker)

Container	Imagem	Função
ssa-db	mariadb	Banco de dados do Zabbix
ssa-zabbix-server	zabbix-server	O "cérebro": roda os checks e dispara as ações
ssa-zabbix-web	zabbix-web-nginx	Interface web de configuração (porta 8081)
ssa-asterisk	andrius/asterisk	Central telefônica (PBX) que faz a ligação
ssa-website	nginx	Site de exemplo, para você testar derrubando-o

O **Linphone** roda no desktop (fora do Docker), como o telefone que recebe a chamada.

Conceitos do Zabbix que valem entender

O Zabbix trabalha em três níveis encadeados:

- **Item** — *o que medir*. Aqui: `web.test.fail` (0 = site OK, diferente de 0 = falhou).
- **Trigger** — *quando isso é problema*. Aqui: `last(web.test.fail) <> 0`.
- **Action** — *o que fazer*. Aqui: chamar o webhook que liga para o telefone.

Como o Zabbix fala com o Asterisk: a ARI

A **ARI** (*Asterisk REST Interface*) é uma API HTTP do Asterisk (porta 8088). O webhook do Zabbix faz um **POST** pedindo a ligação:

```
{
  "endpoint": "PJSIP/1000", // liga para este ramal
  "extension": "alert",    // executa esta extensão...
  "context": "site-alert", // ...neste contexto do dialplan
  "priority": 1
}
```

O Asterisk então segue o *dialplan* (`extensions.conf`): atende, toca o áudio (**Playback**) e desliga.

3. Pré-requisitos

- **Docker** e **Docker Compose** (v2+) instalados
 - Mínimo de **2 GB de RAM** livres
 - Um **softphone** (Linphone) instalado no desktop
 - Sistema Linux (testado em Kali)
-

4. Como subir o serviço

Passo 1 — Clonar / entrar na pasta do projeto

```
cd ~/smart-site-alert
```

Passo 2 — Subir tudo (forma recomendada)

Use o script que sobe a stack e prepara o softphone de uma vez:

```
./start.sh
```

Esse script: 1. Sobe todos os containers (`docker compose up -d`) 2. Espera o Asterisk ficar saudável 3. Reinicia o Linphone limpo, forçando um registro novo do ramal 1000

Por que reiniciar o Linphone junto? O Asterisk roda em container, atrás do NAT do Docker. Quando os containers reiniciam, o registro SIP "morre" e o Linphone (que usa UDP) só re-registraria cerca de 1 hora depois. Reabrir o Linphone força o registro na hora.

Passo 2 (alternativa) — Subir manualmente

```
docker compose up -d      # sobe os containers
docker compose ps         # confere se estão "Up"/"healthy"
```

Depois, abra o Linphone manualmente.

Passo 3 — Acessos

Serviço	Endereço	Credenciais
Zabbix (web)	http://localhost:8081	Admin / zabbix
Asterisk (ARI)	http://localhost:8088/ari	zabbix / ChangeMe_ARI_123
Site de exemplo	http://localhost:8082	—
Ramal SIP	127.0.0.1:5062 (UDP)	1000 / ChangeMe_1000

Passo 4 — Configurar o monitoramento (uma vez só)

A configuração tem 5 partes: **web cenário** → **trigger** → **media type** → **mídia do usuário** → **action**. Você pode criar tudo automaticamente (forma rápida) ou na mão pelo painel (para entender cada peça). Escolha **uma** das duas.

Forma rápida (script via API)

```
python3 scripts/zbx_setup.py
```

Cria tudo de forma idempotente (pode rodar de novo sem duplicar): web cenário "Check site", trigger "Site DOWN", media type Webhook "Asterisk Call" (já habilitado), a mídia do Admin (ramal 1000) e a Action.

Forma manual (pelo painel — passo a passo)

Acesse **http://localhost:8081** e entre com **Admin** / **zabbix**. Faça as 5 etapas na ordem.

4.1 — Web cenário (o que monitora o site)

1. Menu **Data collection** → **Hosts**
2. Na linha do host "**Zabbix server**", clique na coluna **Web** (ou em *Web scenarios*)
3. Botão **Create web cenário** (canto superior direito)
4. Aba **Scenário**: - Name: **Check site** - Update interval: **30s** - Attempts: **1**
5. Aba **Steps** → botão **Add**: - Name: **home** - URL: **http://website** (ou a URL do site real que quer monitorar) - Required status codes: **200** - Clique em **Add** para salvar o passo
6. Clique em **Add** para salvar o cenário

Isso cria automaticamente os itens de coleta, entre eles o `web.test.fail[Check site]` — que vale **0** quando o site responde e **1** quando falha.

4.2 — Trigger (quando considerar que caiu)

1. Ainda em **Data collection** → **Hosts**, na linha do "Zabbix server" clique em **Triggers**
2. Botão **Create trigger**

3. Preencha: - **Name:** `Site DOWN: {HOST.NAME}` - **Severity:** **High** - **Expression:** clique em **Add** e monte, ou cole direto:

```
last(/Zabbix server/web.test.fail[Check site])<>0
```

(lê-se: "se a última medição de falha for diferente de zero → problema") 4. Clique em **Add**

Dica anti-flapping: para só alertar após 3 falhas seguidas, use `min(/Zabbix server/web.test.fail[Check site],#3)<>0`.

4.3 — Media type Webhook (como a ligação é feita)

1. Menu **Alerts** → **Media types**
2. Botão **Create media type**
3. Aba **Media type:** - **Name:** `Asterisk Call` - **Type:** **Webhook** - **Script:** cole o conteúdo do arquivo `scripts/zabbix_webhook.js` - Em **Parameters**, adicione (botão Add em cada um):

Name	Value
<code>ramal</code>	<code>{ALERT.SENDTO}</code>
<code>event_value</code>	<code>{EVENT.VALUE}</code>
<code>ari_url</code>	<code>http://asterisk:8088/ari/channels</code>
<code>ari_user</code>	<code>zabbix</code>
<code>ari_pass</code>	<code>ChangeMe_ARI_123</code>

1. **Enabled** marcado (importante! se ficar desmarcado, a ligação não sai)
2. Clique em **Add**

4.4 — Mídia do usuário (para quem ligar)

1. Menu **Users** → **Users** → clique no usuário **Admin**
2. Aba **Media** → botão **Add:** - **Type:** `Asterisk Call` - **Send to:** `1000` (o ramal que vai tocar) - **When active:** `1-7,00:00-24:00` - **Use if severity:** deixe todas marcadas
3. **Add** e depois **Update**

4.5 — Action (liga a trigger à ligação)

1. Menu **Alerts** → **Actions** → **Trigger actions**
2. Botão **Create action**
3. Aba **Action:** - **Name:** `Ligar quando site cair` - Em **Conditions** → **Add:** **Type** = `Trigger`, e selecione `Site DOWN: Zabbix server`
4. Aba **Operations:** - Em **Operations** → **Add:**
 - **Send to users:** `Admin`
 - **Send only to:** `Asterisk Call`

- *Default operation step duration:* **1h** (evita religar em loop durante a queda)

5. Clique em **Add**

Pronto. A partir daqui, sempre que o site cair, o telefone toca sozinho.

5. Configurar o softphone (Linphone)

Na primeira vez, registre a conta SIP no Linphone:

1. Na tela inicial, escolha "**Third-party SIP account**" (não "Create account")
2. Clique em "**I understand**"
3. Preencha:

Campo	Valor
Username	1000
Password	ChangeMe_1000
Domain	127.0.0.1:5062
Transport	UDP

1. Clique em **Connection**. O status deve ficar verde (registrado).

Atenção ao Transport: tem que ser **UDP**. O padrão do Linphone é TLS, que não funciona com este Asterisk.

6. Como testar

Com tudo no ar e o ramal 1000 registrado:

Teste rápido (sem Zabbix)

```
./scripts/test_call.sh          # liga e toca "site fora do ar"
./scripts/test_call.sh recovery # liga e toca "serviço restabelecido"
```

O Linphone toca. Atenda e ouça a voz.

Teste completo (cadeia automática)

```
docker compose stop website      # derruba o site -> liga em ~30s
# ... o telefone toca sozinho ...
docker compose start website     # restaura o site
```

Você verá: o Zabbix detecta a queda, a Action dispara, o Asterisk liga e o Linphone toca — tudo automático.

7. Comandos do dia a dia

```
cd ~/smart-site-alert

./start.sh                # sobe tudo (containers + Linphone)
docker compose ps         # ver status
docker compose logs -f asterisk # logs do Asterisk em tempo real
docker compose stop website # simular queda
docker compose start website # restaurar
docker compose down        # desligar tudo (config do Zabbix fica salva)
```

8. Resolução de problemas

Problema	Causa provável	Solução
Liga, mas não chega no Linphone	Registro SIP "morto" após restart (NAT do Docker)	Rode <code>./start.sh</code> (reinicia o Linphone e re-registra)
Linphone não registra	Transport errado (TLS) ou porta errada	Use Domain <code>127.0.0.1:5062</code> e Transport UDP
Webhook falha: "Media type disabled"	Media type criado desabilitado	Já corrigido no <code>zbx_setup.py</code> (status habilitado)
Sem áudio no Linphone	Dispositivo de saída errado	Ajuste o dispositivo de áudio nas configs do Linphone

Verificar se o ramal está registrado e "vivo"

```
docker exec ssa-asterisk asterisk -rx "pjsip show contacts"
```

O status deve ser **Avail** (qualificado). Se aparecer `Unavail` ou nada, rode `./start.sh`.

9. Detalhe técnico: o NAT do Docker

A parte mais delicada do projeto é a comunicação entre o **Linphone (no desktop)** e o **Asterisk (em container)**, que passa pelo **NAT do Docker**. Dois cuidados foram aplicados:

1. `qualify_frequency=30` no Asterisk (`pjsip.conf`): o Asterisk envia um "ping" (OPTIONS) ao ramal a cada 30s. Isso mantém o caminho NAT vivo (senão ele expira por inatividade) e detecta quando o ramal cai.
2. `start.sh` reinicia o Linphone junto com a stack, garantindo um registro fresco e qualificado sempre que tudo sobe.

Juntas, essas duas medidas eliminam o sintoma de "ligou mas a chamada não chega".

Smart Site Alert — laboratório de monitoramento com alerta por voz.